University Institute of Engineering

Department of Computer Science & Engineering

COMPUTER ORGANIZATION & ARCHITECTURE

(23CST-204/23ITT-204)

ER. SHIKHA ATWAL

E11186

ASSISTANT PROFESSOR

BE-CSE

INPUT-OUTPUT ORGANIZATION

The input-output subsystem of a computer, referred to as I/O, provides an efficient mode of communication between the central system and the outside environment.

Programs and data must be entered into computer memory for processing and results obtained from computations must be recorded or displayed for the user.

Peripheral Devices

Input or output devices attached to the computer are also called peripherals.

- The display terminal can operate in a single-character mode where all characters entered on the screen through the keyboard are transmitted to the computer simultaneously. In the block mode, the edited text is first stored in a local memory inside the terminal. The text is transferred to the computer as a block of data.
- Printers provide a permanent record on paper of computer output data.
- Magnetic tapes are used mostly for storing files of data.
- Magnetic disks have high-speed rotational surfaces coated with magnetic material.

Input-Output Interface

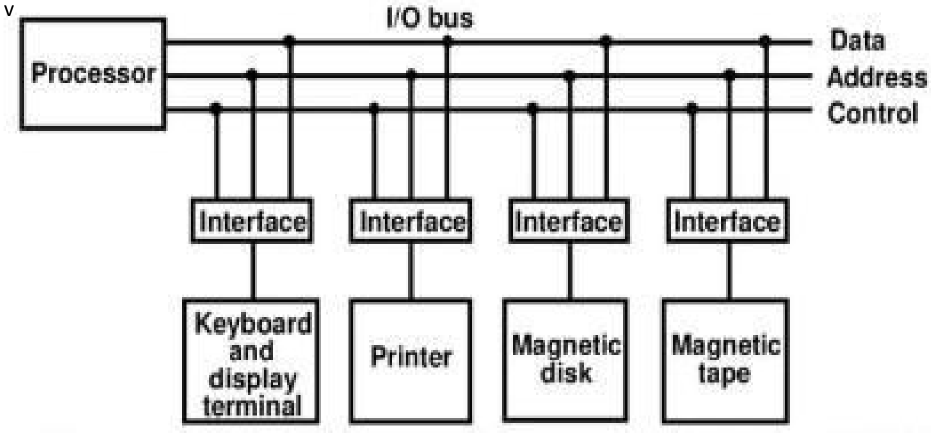
Input-output interface provides a method for transferring information between internal storage and external I/O devices. Peripherals connected to a computer need special communication links for interfacing them with the central processing unit. The major differences are:

1. Peripherals are electromechanical and electromagnetic devices and their manner of operation is different from the operation of the CPU and memory, which are electronic devices. Therefore, a conversion of signal values may be required.
2. The data transfer rate of peripherals is usually slower than the transfer rate of the CPU, and consequently, a synchronization mechanism may be needed.

3. Data codes and formats in peripherals differ from the word format in the CPU and memory.

4. The operating modes of peripherals are different from each other and each must be controlled so as not to disturb the operation of other peripherals connected to the CPU.

A typical communication link between the processor and several peripherals is shown in the figure below.



Connection of I/O bus to input-output devices

The I/O bus consists of data lines, address lines, and control lines. The magnetic disk, printer, and terminal are employed in practically any general-purpose computer. The interface selected responds to the function code and proceeds to execute it.

The function code is referred to as an I/O command and is in essence an instruction that is executed in the interface and its attached peripheral unit.

There are three ways that computer buses can be used to communicate with memory and I/O:

1. Use two separate buses, one for memory and the other for I/O.
2. Use one common bus for both memory and I/O but have separate control lines for each.
3. Use one common bus for memory and I/O with common control lines.

Asynchronous Data Transfer

The internal operations in a digital system are synchronized by means of clock pulses supplied by a common pulse generator. If the registers in the interface share a common clock with the CPU registers, the transfer between the two units is said to be synchronous.

In most cases, the internal timing in each unit is independent from the other in that each uses its own private clock for internal registers. In that case, the two units are said to be asynchronous to each other. This approach is widely used in most computer systems. Asynchronous data transfer between two independent units requires that control signals be transmitted between the communicating units to indicate the time at which data is being transmitted.

There are two ways of achieving this mentioned below:

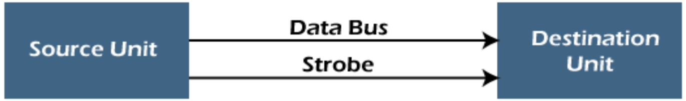
- **Strobe Control:** pulse supplied by one of the units to indicate to the other unit when the transfer has to occur.

- **Handshaking:** The unit receiving the data item responds with another control signal to acknowledge receipt of the data.

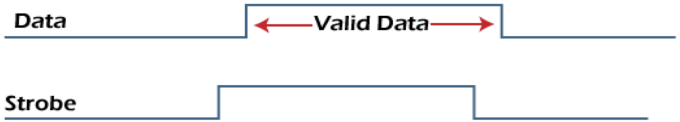
1. Strobe Control Method

The Strobe Control method of asynchronous data transfer employs a single control line to time each transfer. This control line is also known as a strobe, and it may be achieved either by source or destination, depending on which initiate the transfer.

- **Source initiated strobe:** In the below block diagram, you can see that strobe is initiated by source, and as shown in the timing diagram, the source unit first places the data on the data bus.



(a) Block Diagram



(b) Timing Diagram

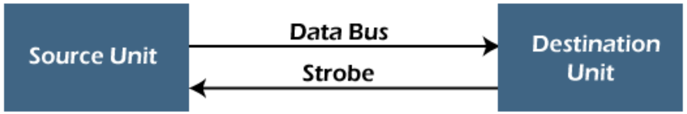
After a brief delay to ensure that the data resolve to a stable value, the source activates a strobe pulse.

The information on the data bus and strobe control signal remains in the active state for a sufficient time to allow the destination unit to receive the data. The destination unit uses a falling edge of strobe control to transfer the contents of a data bus to one of its internal registers.

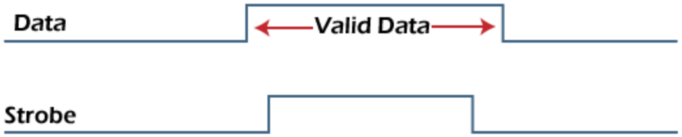
The source removes the data from the data bus after it disables its strobe pulse. Thus, new valid data will be available only after the strobe is enabled again.

In this case, the strobe may be a memory-write control signal from the CPU to a memory unit. The CPU places the word on the data bus and informs the memory unit, which is the destination.

- **Destination initiated strobe:** In the below block diagram, you see that the strobe initiated by destination, and in the timing diagram, the destination unit first activates the strobe pulse, informing the source to provide the data.



(a) Block Diagram



(b) Timing Diagram

- The source unit responds by placing the requested binary information on the data bus. The data must be valid and remain on the bus long enough for the destination unit to accept it.
- The falling edge of the strobe pulse can use again to trigger a destination register. The destination unit then disables the strobe. Finally, and source removes the data from the data bus after a determined time interval. In this case, the strobe may be a memory read control from the CPU to a memory unit. The CPU initiates the read operation to inform the memory, which is a source unit, to place the selected word into the data bus.

2. Handshaking Method

The strobe method has the disadvantage that the source unit that initiates the transfer has no way of knowing whether the destination has received the data that was placed in the bus. Similarly, a destination unit that initiates the transfer has no way of knowing whether the source unit has placed data on the bus.

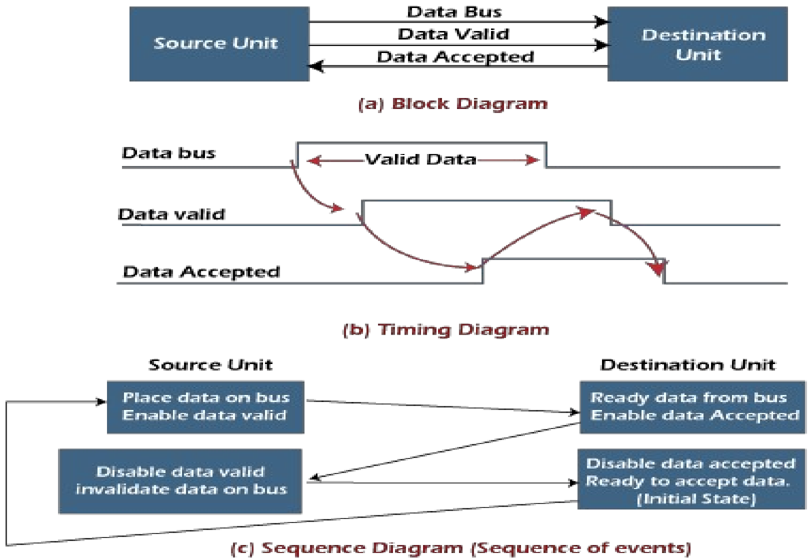
So, this problem is solved by the handshaking method. The handshaking method introduces a second control signal line that replays the unit that initiates the transfer.

In this method, one control line is in the same direction as the data flow in the bus from the source to the destination. The source unit uses it to inform the destination unit whether there are valid data in the bus.

The other control line is in the other direction from the destination to the source. This is because the destination unit uses it to inform the source whether it can accept data. And in it also, the sequence of control depends on the unit that initiates the transfer.

So, it means the sequence of control depends on whether the transfer is initiated by source and destination.

- **Source initiated handshaking:** In the below block diagram, you can see that two handshaking lines are "**data valid**", which is generated by the source unit, and "**data accepted**", generated by the destination unit.



The timing diagram shows the timing relationship of the exchange of signals between the two units.

The source initiates a transfer by placing data on the bus and enabling its data valid signal.

The destination unit then activates the data accepted signal after it accepts the data from the bus.

The source unit then disables its valid data signal, which invalidates the data on the bus.

After this, the destination unit disables its data accepted signal, and the system goes into its initial state.

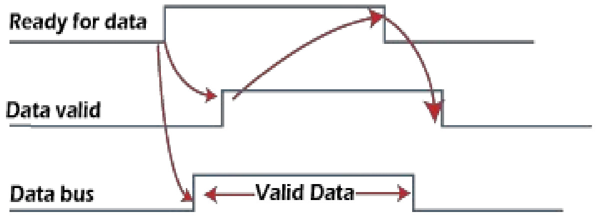
The source unit does not send the next data item until after the destination unit shows readiness to accept new data by disabling the data accepted signal.

This sequence of events described in its sequence diagram, which shows the above sequence in which the system is present at any given time.

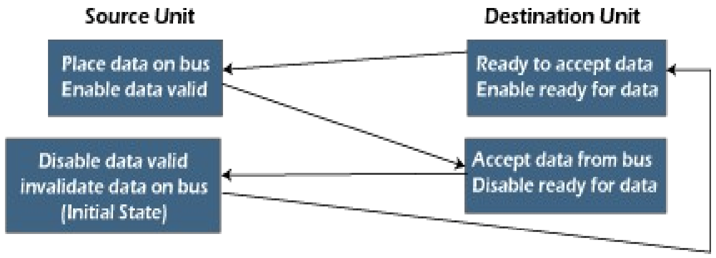
•Destination initiated handshaking:

In the below block diagram, you see that the two handshaking lines are "**data valid**", generated by the source unit, and "**ready for data**" generated by the destination unit. Note that the name of signal data accepted generated by the destination unit has been changed to ready for data to reflect its new meaning.

(a) Block Diagram



(b) Timing Diagram



(c) Sequence Diagram (Sequence of events)

The destination transfer is initiated, so the source unit does not place data on the data bus until it receives a ready data signal from the destination unit. After that, the handshaking process is the same as that of the source initiated. The sequence of events is shown in its sequence diagram, and the timing relationship between signals is shown in its timing diagram. Therefore, the sequence of events in both cases would be identical.

Advantages of Asynchronous Data Transfer

Asynchronous Data Transfer in computer organization has the following advantages, such as:

- It is more flexible, and devices can exchange information at their own pace. In addition, individual data characters can complete themselves so that even if one packet is corrupted, its predecessors and successors will not be affected.
- It does not require complex processes by the receiving device. Furthermore, it means that inconsistency in data transfer does not result in a big crisis since the device can keep up with the data stream. It also makes asynchronous transfers suitable for applications where character data is generated irregularly.

Disadvantages of Asynchronous Data Transfer

There are also some disadvantages of using asynchronous data for transfer in computer organization, such as:

- The success of these transmissions depends on the start bits and their recognition. Unfortunately, this can be easily susceptible to line interference, causing these bits to be corrupted or distorted.
- A large portion of the transmitted data is used to control and identify header bits and thus carries no helpful information related to the transmitted data. This invariably means that more data packets need to be sent.

References

Reference Books:

- J.P. Hayes, “Computer Architecture and Organization”, Third Edition.
- Mano, M., “Computer System Architecture”, Third Edition, Prentice Hall.
- Stallings, W., “Computer Organization and Architecture”, Eighth Edition, Pearson Education.

Text Books:

- Carpinelli J.D,” Computer systems organization &Architecture”, Fourth Edition, Addison Wesley.
- Patterson and Hennessy, “Computer Architecture”, Fifth Edition Morgaon Kauffman.

Other References

- https://tutorialspoint.dev/computer-science/computer-organization-and-architecture/io-interface-interrupt-dma-mode
- Asynchronous Data Transfer in Computer Organization - Javatpoint
- https://www.studytonight.com/computer-architecture/input-output-processor
- Handshaking in Computer architecture (includehelp.com)